Eser Gjoka

Lung Cancer Prediction

Table of contents

01 Exploratory Data Analysis

02 Train Models

03 Test and Evaluate

04 Make Comparison

01

Exploratory Data Analysis

The DataSet

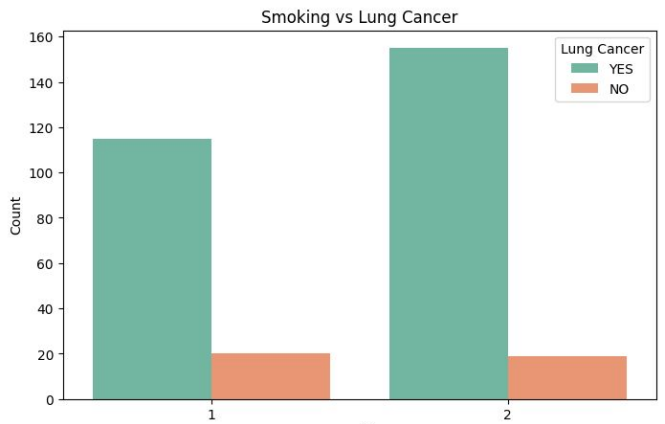
The data set that I got for this project is from kaggle and it is called : ‘Survey Lung Cancer’

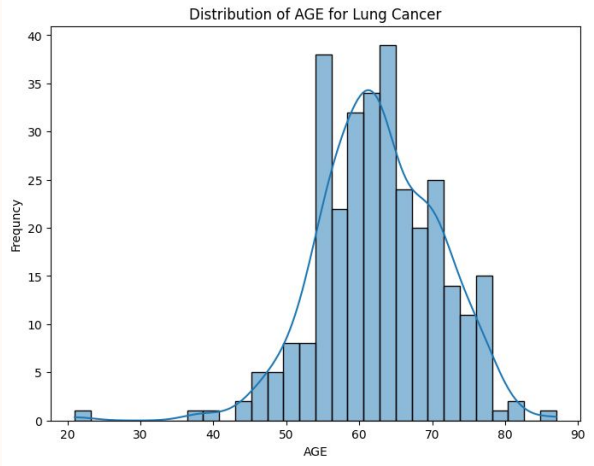
- Minimum 10 Features
- At Least 1 categorical and 1 numerical

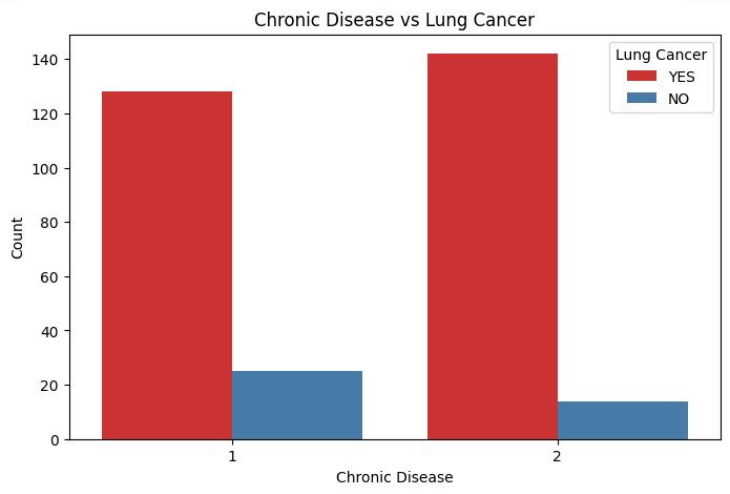
Features include: Age, Smoking, Yellow Fingers, Anxiety, Peer Pressure, Chronic Disease, Fatigue, Allergy, Wheezing, Alcohol Consuming, Coughing, Shortness Of Breath, Swallowing Difficulty, Chest Pain, Lung Cancer, Female, Male

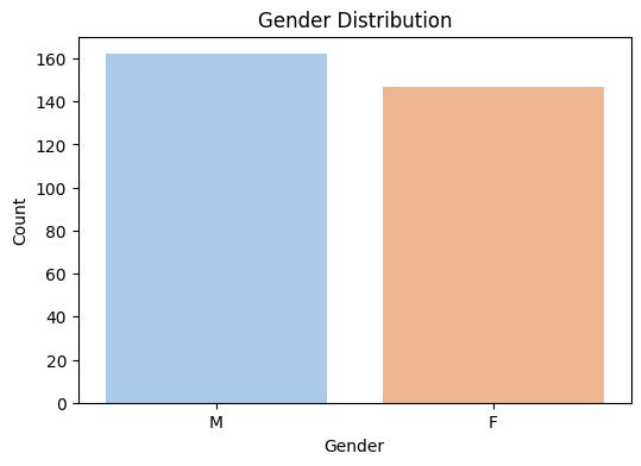
	GENDER	AGE	SMOKING	YELLOW_FINGERS	ANXIETY	PEER_PRESSURE	CHRONIC DISEASE	FATIGUE	ALLERGY	WHEEZING	ALCOHOL CONSUMING	COUGHING	SHORTNESS OF BREATH	SWALLOWING DIFFICULTY	CHEST PAIN	LUNG_CANCER
0	M	69	1	2	2	1	1	2	1	2	2	2	2	2	2	YES
1	M	74	2	1	1	1	2	2	2	1	1	1	2	2	2	YES
2	F	59	1	1	1	2	1	2	1	2	1	2	2	1	2	NO
3	M	63	2	2	2	1	1	1	1	1	2	1	1	2	2	NO
4	F	63	1	2	1	1	1	1	1	2	1	2	2	1	1	NO
5	F	75	1	2	1	1	2	2	2	2	1	2	2	1	1	YES
6	M	52	2	1	1	1	1	2	1	2	2	2	2	1	2	YES
7	F	51	2	2	2	2	1	2	2	1	1	1	2	2	1	YES
8	F	68	2	1	2	1	1	2	1	1	1	1	1	1	1	NO
9	M	53	2	2	2	2	2	1	2	1	2	1	1	2	2	YES
10	F	61	2	2	2	2	2	2	1	2	1	2	2	2	1	YES
11	M	72	1	1	1	1	2	2	2	2	2	2	2	1	2	YES
12	F	60	2	1	1	1	1	2	1	1	1	1	2	1	1	NO
13	M	58	2	1	1	1	1	2	2	2	2	2	2	1	2	YES
14	M	69	2	1	1	1	1	1	2	2	2	2	1	1	2	NO
15	F	48	1	2	2	2	2	2	2	2	1	2	2	2	1	YES
16	M	75	2	1	1	1	2	1	2	2	2	2	2	1	2	YES
17	M	57	2	2	2	2	2	1	1	1	2	1	1	2	2	YES
18	F	68	2	2	2	2	2	2	1	1	1	2	2	1	1	YES
19	F	61	1	1	1	1	2	2	1	1	1	1	2	1	1	NO
20	F	44	2	2	2	2	2	2	1	1	1	1	2	2	1	YES

Graph Visualizations

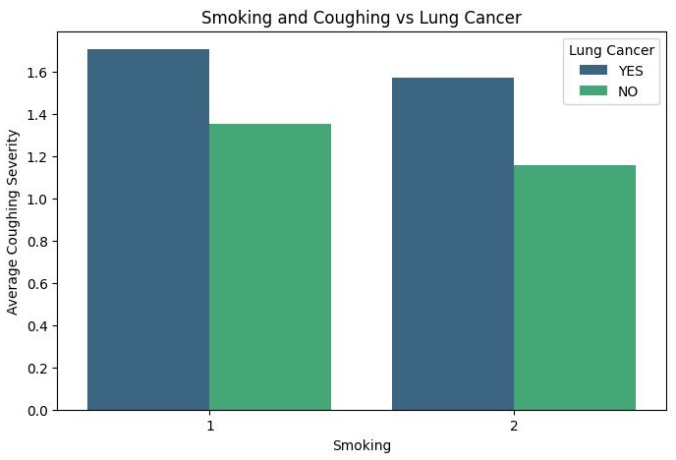


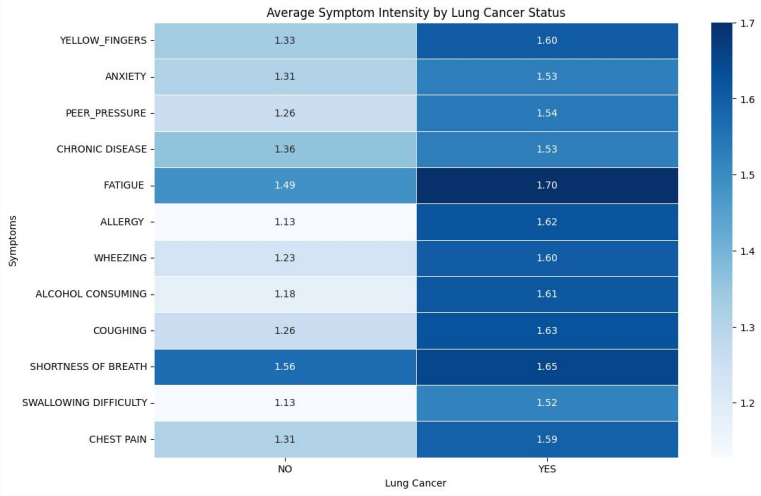


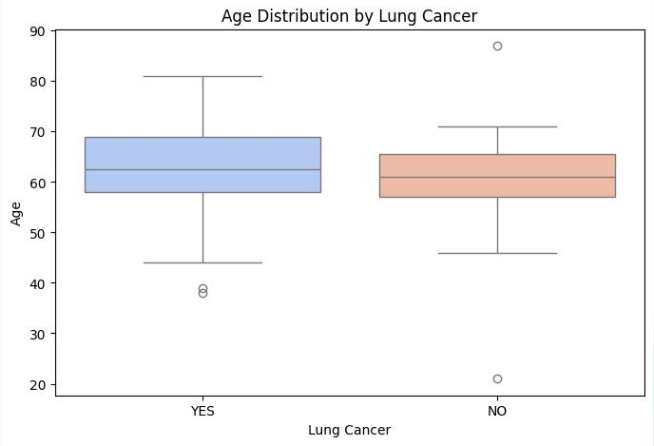


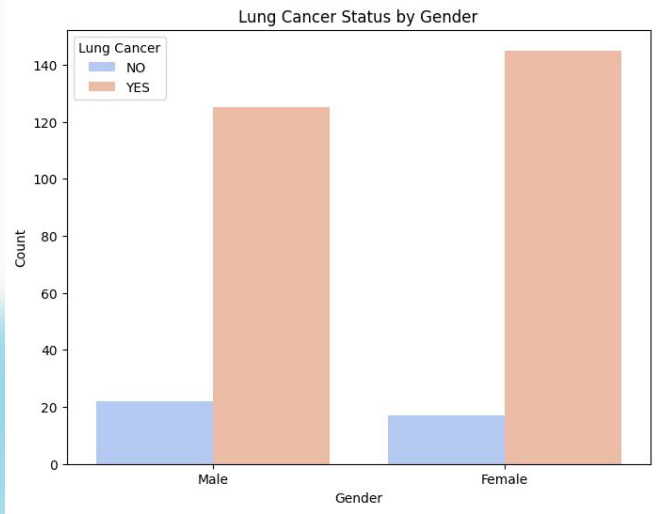


Graph Visualizations









PreProcessing

- This data set was amazing
- No NAN or missing values
  - I did not have to drop any rows due to missing values for replace NAN using average values
- The columns were also all relevant.
  - I did not need to drop irrelevant or unnecessary columns in this data set
- I encoded the numerical columns and the categorical columns then used get dummies

```
numeric_columns = [ 'AGE', 'SMOKING', 'YELLOW_FINGERS', 'ANXIETY',
       'PEER_PRESSURE', 'CHRONIC DISEASE', 'FATIGUE ', 'ALLERGY ', 'WHEEZING',
       'ALCOHOL CONSUMING', 'COUGHING', 'SHORTNESS OF BREATH',
       'SWALLOWING DIFFICULTY', 'CHEST PAIN']

categorical_columns = ['GENDER']
df = pd.get_dummies(df, columns=categorical_columns)
```

```
RangeIndex: 309 entries, 0 to 308
Data columns (total 16 columns):
 #   Column                 Non-Null Count  Dtype
---  ------                 --------------  -----
 0   GENDER                 309 non-null    object
 1   AGE                    309 non-null    int64
 2   SMOKING                309 non-null    int64
 3   YELLOW_FINGERS         309 non-null    int64
 4   ANXIETY                309 non-null    int64
 5   PEER_PRESSURE          309 non-null    int64
 6   CHRONIC DISEASE        309 non-null    int64
 7   FATIGUE                309 non-null    int64
 8   ALLERGY                309 non-null    int64
 9   WHEEZING               309 non-null    int64
 10  ALCOHOL CONSUMING      309 non-null    int64
 11  COUGHING               309 non-null    int64
 12  SHORTNESS OF BREATH    309 non-null    int64
 13  SWALLOWING DIFFICULTY  309 non-null    int64
 14  CHEST PAIN             309 non-null    int64
 15  LUNG_CANCER            309 non-null    object
dtypes: int64(14), object(2)
memory usage: 38.8+ KB
```

```
print(df['AGE'].describe())

count    309.000000
mean      62.673139
std        8.210301
min       21.000000
25%       57.000000
50%       62.000000
75%       69.000000
max       87.000000
Name: AGE, dtype: float64
```

PreProcessing

- Then standardized and also normalized the dataframe
- Set X and y and printed their respective shapes
- Normalization and Standardization ensures all features were on a similar scale, reducing bias in models

```
X = df.drop(columns=['LUNG_CANCER'])
y = df['LUNG_CANCER']
```

```
X.shape
```

```
(309, 16)
```

```
y.shape
```

```
(309,)
```

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn import preprocessing
from sklearn.model_selection import train_test_split
from sklearn.impute import SimpleImputer
original = df[numeric_columns]
# Standardize dataframe and return as an array
standardizedArray = preprocessing.scale(original)

# Convert standardized array to dataframe 'standardized'
standardized = pd.DataFrame(standardizedArray, columns=numeric_columns)
```

```
# Normalize dataframe and return as an array
normalizedArray = preprocessing.MinMaxScaler().fit_transform(df[numeric_columns])

# Convert normalized array to dataframe 'normalized'
normalized = pd.DataFrame(normalizedArray, columns=numeric_columns)
normalized
```

	AGE	SMOKING	YELLOW_FINGERS	ANXIETY	PEER_PRESSURE	CHRONIC DISEASE	FATIGUE	ALLERGY	WHEEZING	ALCOHOL CONSUMING	COUGHING	SHORTNESS OF BREATH	SWALLOWING DIFFICULTY	CHEST PAIN
0	0.727273	0.0	1.0	1.0	0.0	0.0	1.0	0.0	1.0	1.0	1.0	1.0	1.0	1.0
1	0.803030	1.0	0.0	0.0	0.0	1.0	1.0	1.0	0.0	0.0	0.0	1.0	1.0	1.0
2	0.575758	0.0	0.0	0.0	1.0	0.0	1.0	0.0	1.0	0.0	1.0	1.0	0.0	1.0
3	0.636364	1.0	1.0	1.0	0.0	0.0	0.0	0.0	0.0	1.0	0.0	0.0	1.0	1.0
4	0.636364	0.0	1.0	0.0	0.0	0.0	0.0	0.0	1.0	0.0	1.0	1.0	0.0	0.0
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...
304	0.530303	0.0	0.0	0.0	1.0	1.0	1.0	0.0	0.0	1.0	1.0	1.0	1.0	0.0
305	0.742424	1.0	0.0	0.0	0.0	0.0	1.0	1.0	1.0	1.0	1.0	1.0	0.0	1.0
306	0.560606	1.0	0.0	0.0	0.0	0.0	0.0	1.0	1.0	1.0	1.0	0.0	0.0	1.0
307	0.696970	1.0	0.0	1.0	0.0	0.0	1.0	1.0	0.0	1.0	1.0	1.0	0.0	1.0
308	0.621212	0.0	0.0	0.0	1.0	0.0	1.0	1.0	1.0	1.0	0.0	0.0	1.0	0.0

02

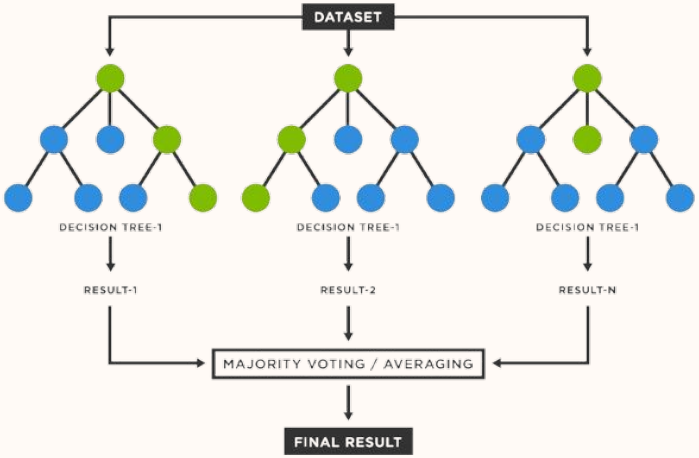
Train Models

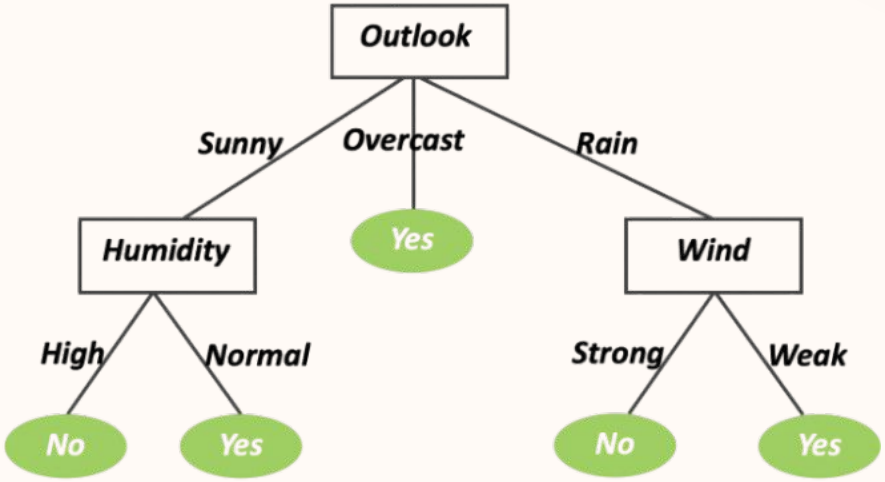
Training Models

- The goal is to predict whether or not a person has lung cancer
- I chose to use Classification models instead of Regression
- I used a minimum of three features where one of these features is categorical
- Optimization algorithms used:
  - Random Forest, Decision Tree, Logistic Regression, SVC, KNN
- Data was split the into 80% training and 20% testing to evaluate model performance

Training Models

- The 2 models I chose are Random Forest and Decision Tree Classifier
  - I chose to explain these 2 models because they were the most accurate and yielded the best results





Random Forest

- Description: Random Forest is an ensemble learning method that builds multiple decision trees during training. It outputs the class that is the majority vote among the individual trees.

- Why Used: Random Forest was chosen for its ability to handle feature interactions and manage imbalanced datasets effectively. Less influenced by outliers

Random Forest

- Random Forest is a version of ensemble learning
  - Ensemble learning is when we take multiple algorithms or the same algorithm multiple times and put them together to make something much more powerful than the original.
    - The wisdom of the crowds

Random Forest

- Random Forest Steps
- • Step 1: Pick at random K data points from the training set
- • Step 2: build a Decision Tree based on these K data points
- • Step 3: choose the number of trees you want to build and repeat steps 1 & 2
- • Step 4: when trying to predict a new data point, make each one of your trees predict the category to which the data point belongs, and assign the new data point to the category that wins the majority vote

Decision Tree Classifier

Description: The Decision Tree Classifier is a supervised algorithm that splits data into subsets based on feature values, forming a tree where nodes represent features, branches are decision rules, and leaves are class labels.

Why Used: Easy to understand and Interpret. Provides a visual representation of the decision making process. Can Handle large data sets. Provide insights into feature importance, helping identify which variables contribute most to predictions.

Decision Tree Classifier

Steps in building the Decision Tree

- We want to know which of the attributes gives the maximum information out of the available attributes (independent variables/features).
- For that we will need to calculate the information gain for every attribute.
- The attribute that has the maximum information gain will be considered as a root node.

You then calculate the information gain for each attribute

03

Test and Evaluate

Test and Evaluate

Random Forest Classifier

- Mathematical Formulas used:
  - R^2
    - 1 – residual sum of square(SSR) / total sum of squares(SST).
  - Accuracy
    - Accuracy=Total Number of Predictions/Number of Correct Predictions
      - (TP+TN)/(TP+TN+FP+FN)
  - ConfusionMatrix
    - (TP+TN)/(TP+FP+TN+FN)
    - Where: TP: True positives TN: True negatives FP: False positives FN: False negative
  - Classification Report
    - Precision = TP/(TP+FP)
    - Recall = TP/(TP+FN)
    - F1-Score = 2*(Precision*Recall)/(Precision+Recall)
    - Support = The number of true instances for each class in the dataset.

```
rf = RandomForestClassifier()
rf.fit(X_train, y_train)
```

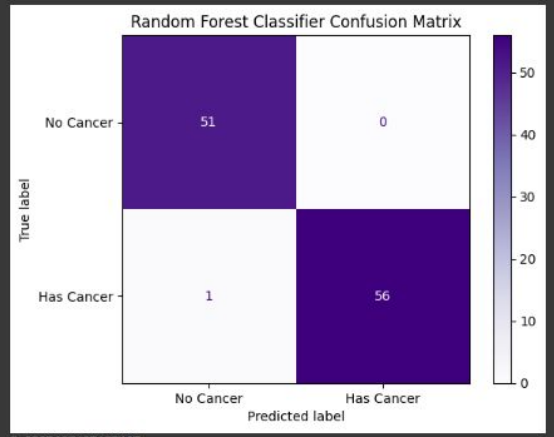
```
RandomForestClassifier
RandomForestClassifier()
```

```
rf_accuracy= evaluate_model(rf, X_test, y_test, 'Random Forest Classifier')
rf_accuracy
```

```
Random Forest Classifier Accuracy: 0.99
Random Forest Classifier Classification Report:
              precision    recall  f1-score   support

          NO       0.98      1.00      0.99        51
         YES       1.00      0.98      0.99        57

    accuracy                           0.99       108
   macro avg       0.99      0.99      0.99       108
weighted avg       0.99      0.99      0.99       108
```




```
0.9907407407407407
```

```
Random Forest Classifier R^2: 0.9628
0.9628482972136223
```

Test and Evaluate

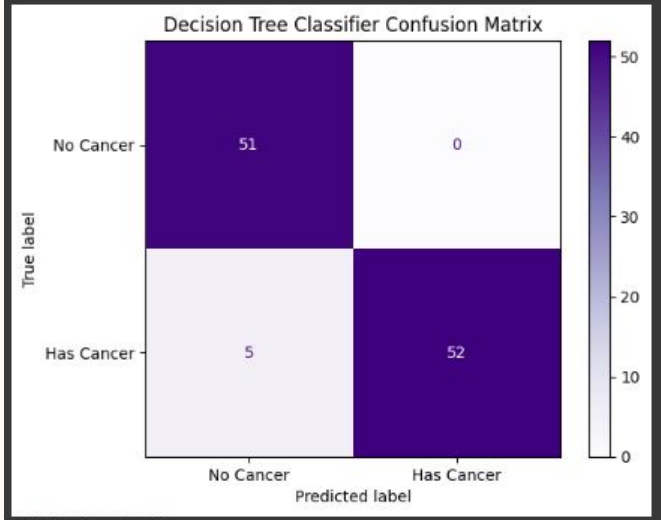
Decision Tree Classifier

- Mathematical Formulas used:
  - R^2
    - 1 – residual sum of square(SSR) / total sum of squares(SST).
  - Accuracy
    - Accuracy=Total Number of Predictions/Number of Correct Predictions
      - (TP+TN)/(TP+TN+FP+FN)
  - ConfusionMatrix
    - (TP+TN)/(TP+FP+TN+FN)
    - Where: TP: True positives TN: True negatives FP: False positives FN: False negative
  - Classification Report
    - Precision = TP/(TP+FP)
    - Recall = TP/(TP+FN)
    - F1-Score = 2*(Precision*Recall)/(Precision+Recall)
    - Support = The number of true instances for each class in the dataset.

```
Decision Tree Classifier Accuracy: 0.95
Decision Tree Classifier Classification Report:
              precision    recall  f1-score   support

          NO       0.91      1.00      0.95        51
         YES       1.00      0.91      0.95        57

    accuracy                           0.95       108
   macro avg       0.96      0.96      0.95       108
weighted avg       0.96      0.95      0.95       108
```




```
0.9537037037037037
```

```
Decision Tree Classifier R^2: 0.8514
0.8513931888544891
```

Test and Evaluate

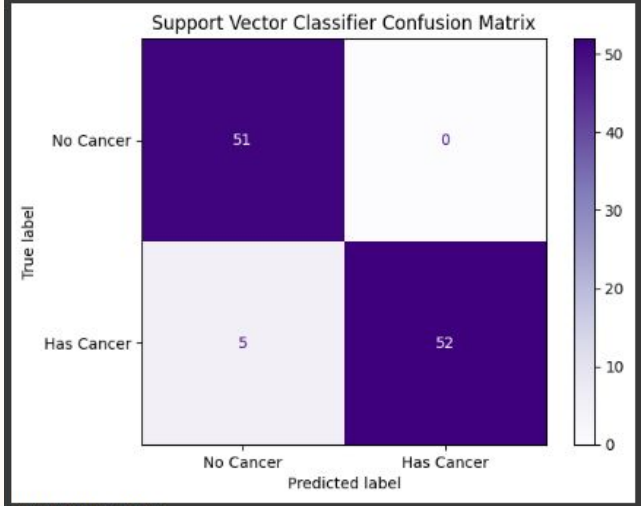
SVC

- Mathematical Formulas used:
  - R^2
    - 1 – residual sum of square(SSR) / total sum of squares(SST).
  - Accuracy
    - Accuracy=Total Number of Predictions/Number of Correct Predictions
      - (TP+TN)/(TP+TN+FP+FN)
  - ConfusionMatrix
    - (TP+TN)/(TP+FP+TN+FN)
    - Where: TP: True positives TN: True negatives FP: False positives FN: False negative
  - Classification Report
    - Precision = TP/(TP+FP)
    - Recall = TP/(TP+FN)
    - F1-Score = 2*(Precision*Recall)/(Precision+Recall)
    - Support = The number of true instances for each class in the dataset.

```
Support Vector Classifier Accuracy: 0.95
Support Vector Classifier Classification Report:
              precision    recall  f1-score   support

          NO       0.91      1.00      0.95        51
         YES       1.00      0.91      0.95        57

    accuracy                           0.95       108
   macro avg       0.96      0.96      0.95       108
weighted avg       0.96      0.95      0.95       108
```




```
0.9537037037037037
```

```
Support Vector Classifier R^2: 0.8142
0.8142414860681114
```

Test and Evaluate

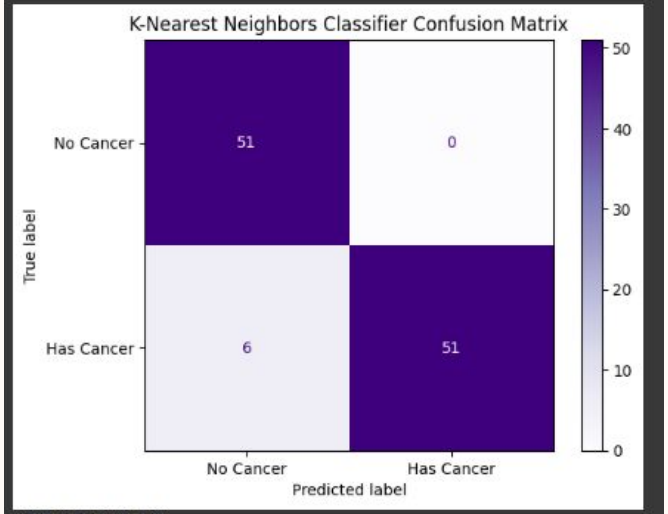
KNN

- Mathematical Formulas used:
  - R^2
    - 1 – residual sum of square(SSR) / total sum of squares(SST).
  - Accuracy
    - Accuracy=Total Number of Predictions/Number of Correct Predictions
      - (TP+TN)/(TP+TN+FP+FN)
  - ConfusionMatrix
    - (TP+TN)/(TP+FP+TN+FN)
    - Where: TP: True positives TN: True negatives FP: False positives FN: False negative
  - Classification Report
    - Precision = TP/(TP+FP)
    - Recall = TP/(TP+FN)
    - F1-Score = 2*(Precision*Recall)/(Precision+Recall)
    - Support = The number of true instances for each class in the dataset.

```
K-Nearest Neighbors Classifier Accuracy: 0.94
K-Nearest Neighbors Classifier Classification Report:
              precision    recall  f1-score   support

          NO       0.89      1.00      0.94        51
         YES       1.00      0.89      0.94        57

    accuracy                           0.94       108
   macro avg       0.95      0.95      0.94       108
weighted avg       0.95      0.94      0.94       108
```




```
0.9444444444444444
```

```
K-Nearest Neighbors Classifier R^2: 0.7771
0.7770897832817337
```

04

Make Comparisons

Make Comparisons

1. Random Forest Classifier
Performance: Accuracy: 99% R^2: 0.9628
Precision, Recall, and F1-Score:
For "No Cancer": High precision (98%) and perfect recall (100%).
For "Has Cancer": Perfect precision (100%) and high recall (98%).
Confusion Matrix: Misclassified only 1 sample as false negative ("Has Cancer" predicted as "No Cancer").

Strengths:
Robustness: Handles non linear relationships and feature interactions well.
Outlier Handling: Less sensitive to outliers due to averaging across multiple trees.
Feature Importance: Provides insights into which features were most influential.

Weaknesses:
Overfitting Risk: Though mitigated by bagging, deeper trees can still overfit
Complexity: Requires more computational resources than simpler models

Make Comparisons

Decision Tree Classifier
Performance: Accuracy: 95% R^2: 0.8514
Precision, Recall, and F1-Score:
For "No Cancer": Precision (91%), Recall (100%), and F1-Score (95%).
For "Has Cancer": Perfect precision (100%), Recall (91%), and F1-Score (95%).
Confusion Matrix:
Misclassified 5 samples as false negatives ("Has Cancer" predicted as "No Cancer").

Strengths:
Interpretability: Decision trees are easy to visualize and interpret, showing clear decision paths.
Speed: Relatively faster to train compared to ensemble models.
Non-linear Relationships: Captures non-linear relationships between features and the target.

Weaknesses:
Overfitting: Tends to overfit the training data, leading to reduced generalization performance.
Outlier Sensitivity: More sensitive to outliers, which can skew the splits.

Make Comparisons

Discussion

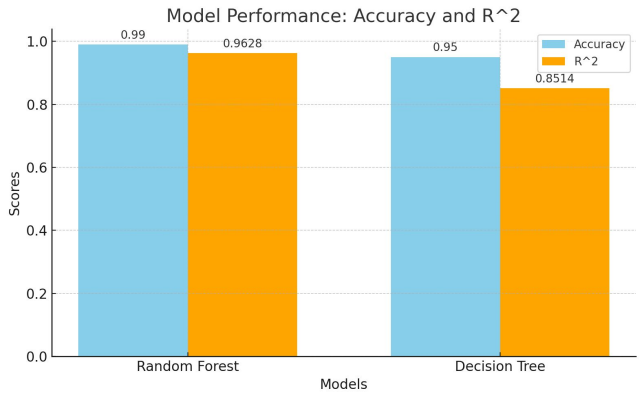
Why Random Forest Outperforms Decision Tree:

Random Forest reduces overfitting by averaging multiple trees (bagging), which helps generalize better on unseen data. This is evident from its higher accuracy (99% vs. 95%) and R^2 score (0.9628 vs. 0.8514).
Random Forest effectively handled feature noise and potential outliers, leading to fewer misclassifications (1 vs. 5 false negatives).

Strength of Decision Tree:
The Decision Tree model is simpler, interpretable, and computationally efficient. However, it is prone to overfitting and less robust when handling noisy data or feature interactions.

Weakness of Decision Tree:
It failed to generalize as well as the Random Forest, leading to more false negatives in the confusion matrix.

Make Comparisons

Conclusion

- Best Performing Model: Random Forest, due to its superior accuracy and ability to generalize across the dataset.
- Random Forest outperformed other models with 99% accuracy and the highest R^2
- The Random Forest classifier emerged as the best performing model for lung cancer prediction due to its accuracy, robustness on unseen data.
- Use Random Forest for high-stakes scenarios where accuracy is critical.
- Use Decision Tree when interpretability and speed are more important.